INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, SONEPAT

भारतीय सूचना प्रौद्योगिकी संस्थान, सोनीपत

Computer Organization Lab (CSC308)

Submitted By:	**Submitted To:**
Name – Aditya Bhardwaj	Dr. Ashish Ranjan
Roll no - 12311093	
Sem – 3rd	
Class - CSE B	

Sr. No.	Topic	Page	Sign.
1	Introduction to gates	3	
2	Ripple carry adder	4	
3	CLA Adder	7	
4	Registers and counters	7	
5	Wallace Tree Adder	9	
6	Combinational Multipliers	10	
7	Booth's Multiplier	12	
8	ALU	15	
9	Memory Design	15	
10	ACD	16	
11	DMCD	17	
12	CPU	17	
13	Mathematical Expressions	18	
14	File Operations	19	

EXPERIMENT -1

Introduction to Gates

- **Theory**: Logic gates are the building blocks of digital circuits. They operate on one or more binary inputs to produce a binary output based on a predefined logic function. The basic gates include:
    - **AND**: Output is 1 if both inputs are 1. ○ **OR**:

      Output is 1 if at least one input is 1. ○ **NOT**:

      Output is the inverse of the input. ○ **XOR**:

      Output is 1 if the inputs are different.
    - **NAND, NOR**: Inverse of AND and OR, respectively.

Code:

```
public class LogicGates {

    // AND Gate: Returns true if both inputs are true
    public static boolean AND(boolean a, boolean b) {
        return a && b;
    }

    // OR Gate: Returns true if at least one input is true
    public static boolean OR(boolean a, boolean b) {
        return a || b;
    }

    // NOT Gate: Returns the negation of the input
    public static boolean NOT(boolean a) {
        return !a;
    }

    // XOR Gate: Returns true if the inputs are different
    public static boolean XOR(boolean a, boolean b) {
        return a ^ b;  // XOR is the same as a != b for Boolean values
    }

    // Main method to test the logic gates
    public static void main(String[] args) {
        boolean inputA = true;
        boolean inputB = false;

        System.out.println("Input A: " + inputA + ", Input B: " + inputB);
        System.out.println("AND Gate: " + AND(inputA, inputB));
        System.out.println("OR Gate: " + OR(inputA, inputB));
        System.out.println("NOT Gate (A): " + NOT(inputA));
        System.out.println("NOT Gate (B): " + NOT(inputB));
        System.out.println("XOR Gate: " + XOR(inputA, inputB));
    }
}
```

Output:

```
adityabhardwaj@Adityas-Air JAVA %  /usr/bin/env /
  -cp /Users/adityabhardwaj/Library/Application\ S
 gicGates
 Input A: true, Input B: false
 AND Gate: false
 OR Gate: true
 NOT Gate (A): false
 NOT Gate (B): true
 XOR Gate: true
adityabhardwaj@Adityas-Air JAVA %
```

EXPERIMENT -2

Ripple Carry Adder

- **Theory**: A Ripple Carry Adder (RCA) is a simple circuit for adding two binary numbers. It consists of multiple full adders connected in series, where the carry output of one adder is used as the carry input for the next. The "ripple" effect occurs because the carry bit propagates from one stage to the next, making it slower as the number of bits increases.

Code:

```java
public class RippleCarryAdder {


        // Full Adder method (one bit adder)

        public static class FullAdder {

            // The FullAdder takes three inputs: A, B, and the carry-in (Cin)

            // and outputs the sum and carry-out (Cout)

            public static int[] fullAdder(int A, int B, int Cin) {

                int sum = A ^ B ^ Cin;        // Sum is A XOR B XOR Cin

                int Cout = (A & B) | (Cin & (A ^ B));  // Carry-out is (A AND B) OR (Cin AND (A XOR B))

                return new int[]{sum, Cout};   // Return the sum and carry-out as an array

            }

        }



        // Ripple Carry Adder method for two binary numbers

        public static int[] rippleCarryAdder(int[] A, int[] B) {

            int n = A.length; // Assuming both A and B have the same length

            int[] sum = new int[n + 1]; // Resultant sum array (one extra bit for the final carry-out)

            int carry = 0;



            // Adding corresponding bits from A and B using full adders

            for (int i = 0; i < n; i++) {
```

```java
        int[] result = FullAdder.fullAdder(A[i], B[i], carry);

        sum[i] = result[0];  // The sum bit is at index i

        carry = result[1];    // The carry is passed to the next bit

    }

    sum[n] = carry;  // The final carry goes into the most significant bit (MSB)



    return sum;  // Return the sum array

}



// Method to print the binary number (for display)

public static void printBinary(int[] bin) {

    for (int i = bin.length - 1; i >= 0; i--) {

        System.out.print(bin[i]);

    }

    System.out.println();

}



// Main method to test the Ripple Carry Adder

public static void main(String[] args) {

    // Example: 4-bit numbers A and B

    int[] A = {1, 0, 1, 0};  // A = 10 in decimal

    int[] B = {1, 1, 0, 1};  // B = 13 in decimal



    System.out.print("A = ");

    printBinary(A);

    System.out.print("B = ");

    printBinary(B);
```

```java
        int[] sum = rippleCarryAdder(A, B);  // Perform the addition

        System.out.print("Sum = ");

        printBinary(sum);  // Print the result



        // Optionally, print the decimal result

        System.out.println("Decimal Sum = " + binaryToDecimal(sum));

    }



    // Method to convert binary array to decimal value

    public static int binaryToDecimal(int[] bin) {

        int decimal = 0;

        int power = 1;

        for (int i = 0; i < bin.length; i++) {

            decimal += bin[i] * power;

            power *= 2;

        }

        return decimal;

    }

}
```

Output:

```
adityabhardwaj@Adityas-Air JAVA %  /usr/bir
ExceptionMessages -cp /Users/adityabhardwaj
t.java/jdt_ws/JAVA_db575eb7/bin RippleCarry
A = 0101
B = 1011
Sum = 10000
Decimal Sum = 16
adityabhardwaj@Adityas-Air JAVA %
```

EXPERIMENT -3

Carry-Look-Ahead Adder

Theory: A Carry-Look-Ahead Adder (CLA) improves on the RCA by calculating carry bits more quickly. It uses two concepts—carry generate (G) and carry propagate (P)—to determine carry values in parallel, rather than waiting for them to ripple through each bit. This significantly reduces the addition time, making it faster for large numbers.

Code:

```
#include <iostream> #include <vector>
using namespace std;

int generate(int a, int b) { return a & b; }
int propagate(int a, int b) { return a ^ b; }
vector<int> carryLookaheadAdder(vector<int>& a, vector<int>& b) {     int n = a.size();     vector<int> sum(n);
   vector<int> carry(n + 1, 0);

   for (int i = 0; i < n; ++i) {         sum[i] = propagate(a[i], b[i]) ^ carry[i];         carry[i + 1] = generate(a[i], b[i]) | (propagate(a[i], b[i]) & carry[i]);
   }
   return sum;
}

int main() {
   vector<int> a = {1, 0, 1}; // Binary 5     vector<int> b = {1, 1, 1}; // Binary 7

   vector<int> result = carryLookaheadAdder(a, b);

   cout << "Sum: ";     for (int i = 0; i < result.size(); ++i) {         cout << result[i];
   }
   cout << endl;     return 0;
}
```

Output:

```
Sum: 001
```

EXPERIMENT -4

Registers and Counters

- **Theory:** A register is a small, fast storage unit used to hold data temporarily in a CPU.

  A counter is a specific type of register that increments or decrements its value with each clock cycle, often used for tracking events or as part of control circuits. Counters are typically built using flip-flops.

Code:

```
public class RegisterAndCounter {

    // Base class for Register
    public static class Register {
        protected int value; // The value stored in the register

        // Constructor to initialize register with an initial value
        public Register(int initialValue) {
            value = initialValue;
        }

        // Method to load a value into the register
        public void load(int value) {
            this.value = value;
        }

        // Method to read the value of the register
        public int read() {
            return value;
        }

        // Method to clear the register (set value to 0)
        public void clear() {
            value = 0;
        }

        // Method to increment the value of the register
        public void increment() {
            value++;
        }

        // Method to decrement the value of the register
        public void decrement() {
            value--;
        }

        // Method to print the value stored in the register
        public void print() {
            System.out.println("Register value: " + value);
        }
    }

    // Counter class that extends Register
    public static class Counter extends Register {

        // Constructor to initialize the counter with an initial value
        public Counter(int initialValue) {
            super(initialValue);
        }

        // Method to increment the counter by 1
        @Override
        public void increment() {
            value++;
        }

        // Method to decrement the counter by 1
        @Override
        public void decrement() {
            value--;
        }

        // Method to reset the counter to a specified value
        public void reset(int resetValue) {
            value = resetValue;
        }

        // Method to print the counter value
        @Override
        public void print() {
            System.out.println("Counter value: " + value);
        }
    }

    public static void main(String[] args) {
        // --- Test Register Class ---
        System.out.println("Testing Register:");
        Register reg = new Register(10); // Initialize register with value 10
        reg.print(); // Output: Register value: 10

        reg.increment(); // Increment register value by 1
        reg.print(); // Output: Register value: 11

        reg.load(25); // Load a new value into the register
```

```
        reg.print(); // Output: Register value: 25

        reg.clear(); // Clear the register (set value to 0)
        reg.print(); // Output: Register value: 0

        // --- Test Counter Class ---
        System.out.println("\nTesting Counter:");
        Counter counter = new Counter(0); // Initialize counter with 0
        counter.print(); // Output: Counter value: 0

        counter.increment(); // Increment counter value by 1
        counter.print(); // Output: Counter value: 1

        counter.increment(); // Increment counter value by 1
        counter.print(); // Output: Counter value: 2

        counter.decrement(); // Decrement counter value by 1
        counter.print(); // Output: Counter value: 1

        counter.reset(5); // Reset counter to value 5
        counter.print(); // Output: Counter value: 5
    }
}
```

Output:

```
● adityabhardwaj@Adityas-Air JAVA %
  ExceptionMessages -cp /Users/adit
  t.java/jdt_ws/JAVA_db575eb7/bin R
  Testing Register:
  Register value: 10
  Register value: 11
  Register value: 25
  Register value: 0

  Testing Counter:
  Counter value: 0
  Counter value: 1
  Counter value: 2
  Counter value: 1
  Counter value: 5
○ adityabhardwaj@Adityas-Air JAVA %
```

EXPERIMENT -5

Wallace Tree Adder

- **Theory:** The Wallace Tree Adder is a hardware multiplier designed to speed up binary multiplication. It reduces the number of partial product terms in parallel using a tree structure to carry out summation.

  This reduces the delay compared to other methods like the traditional shift-and-add.

Code:

```
#include
<iostream>
#include <vector>
using namespace
std; // Full adder
function
int fullAdder(int a, int b, int c, int& carryOut) {     int sum
= a + b + c;  // Sum of the bits
    carryOut = sum / 2;    // Carry out is 1 if sum is 2 or more     return sum % 2;
// Return sum mod 2 (0 or 1)
}
```

```
int main() {
   int A = 3, B = 5; // Binary 0011 * 0101
vector<int> partialProducts(4);     // Calculate
partial products     partialProducts[0] = A & 1 ? B
: 0;     partialProducts[1] = A & 2 ? (B << 1) : 0;
partialProducts[2] = A & 4 ? (B << 2) : 0;
partialProducts[3] = A & 8 ? (B << 3) : 0;
   // Use Wallace tree for final addition (using full adder)     int sum = 0,
carry = 0;
   // Sum the partial products bit by bit using fullAdder     for (int
i = 0; i < 4; ++i) {         int bitSum = 0;         int currentCarry =
carry;         for (int j = 0; j < 4; ++j) {
          // Use full adder for each bit position
          bitSum = fullAdder(partialProducts[i] >> j & 1, sum >> j & 1, currentCarry, currentCarry);            sum |= (bitSum << j);
       }
       carry = currentCarry;
   }
   cout << "Result: " << sum << endl; // Output will be 15 for 3 * 5

   return 0;
}
```

Output:

```
Result: 15
```

EXPERIMENT -6

Combinational Multipliers

- **Theory:** A combinational multiplier is a circuit that multiplies two binary numbers without using sequential logic (i.e., no clock). It typically uses AND gates to generate partial products and adders (such as ripple-carry adders) to sum these partial products. The simplest form is a direct binary multiplier.

```
public class ArrayMultiplier {

   // Method to add two binary numbers
   public static int[] addBinary(int[] A, int[] B) {
      int n = Math.max(A.length, B.length);
      int[] sum = new int[n + 1]; // Extra bit for carry
      int carry = 0;

      // Start adding from the least significant bit
      for (int i = 0; i < n; i++) {
         int aBit = i < A.length ? A[i] : 0;
         int bBit = i < B.length ? B[i] : 0;
         int total = aBit + bBit + carry;
         sum[i] = total % 2; // Sum bit
         carry = total / 2;   // Carry bit
      }

      // Handle the final carry bit
      sum[n] = carry;
      return sum;
   }
```

```java
    // Method to multiply two binary numbers using an array multiplier
    public static int[] arrayMultiplier(int[] A, int[] B) {
        int n = A.length;
        int[] result = new int[2 * n]; // Result can be at most 2 * n bits for two n-bit numbers

        // Generate partial products
        int[][] partialProducts = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                partialProducts[i][j] = A[i] & B[j];  // AND operation for each bit pair
            }
        }

        // Add the partial products using a binary adder
        int[] currentSum = new int[2 * n]; // Temporary array for addition
        for (int row = 0; row < n; row++) {
            int[] shiftedPartialProduct = new int[2 * n];  // Shift the partial product
            for (int col = 0; col < n; col++) {
                shiftedPartialProduct[row + col] = partialProducts[row][col];
            }

            // Add the shifted partial product to the current sum
            currentSum = addBinary(currentSum, shiftedPartialProduct);
        }

        return currentSum;
    }

    // Method to print binary number
    public static void printBinary(int[] bin) {
        for (int i = bin.length - 1; i >= 0; i--) {
            System.out.print(bin[i]);
        }
        System.out.println();
    }

    // Main method to test the array multiplier
    public static void main(String[] args) {
        // Test with two 4-bit numbers (e.g., 6 and 3)
        int[] A = {1, 1, 0, 0};  // A = 6 in decimal
        int[] B = {1, 0, 0, 1};  // B = 3 in decimal

        System.out.print("A = ");
        printBinary(A);
        System.out.print("B = ");
        printBinary(B);

        // Perform multiplication using Array Multiplier
        int[] product = arrayMultiplier(A, B);

        System.out.print("Product = ");
        printBinary(product);  // Output the binary product

        // Optionally, print the decimal result for verification
        int decimalProduct = binaryToDecimal(product);
        System.out.println("Decimal Product = " + decimalProduct);
    }

    // Method to convert binary array to decimal value
    public static int binaryToDecimal(int[] bin) {
        int decimal = 0;
        int power = 1;
        for (int i = 0; i < bin.length; i++) {
            decimal += bin[i] * power;
            power *= 2;
        }
        return decimal;
    }
}
```

Output:

```
adityabhardwaj@Adityas-Air JAVA %
ExceptionMessages -cp /Users/adityа
t.java/jdt_ws/JAVA_db575eb7/bin Ar
A = 0011
B = 1001
Product = 000000011011
Decimal Product = 27
adityabhardwaj@Adityas-Air JAVA %
```

EXPERIMENT -7

Booth's Multiplier

Theory:

Booth's algorithm is a technique used for efficient multiplication of binary numbers, particularly useful when multiplying signed numbers. It reduces the number of operations required to compute the product by encoding both positive and negative numbers efficiently. The algorithm handles pairs of bits in the multiplier to simplify the number of add/subtract operations needed.

Code:

```
public class BoothMultiplier {

    // Method to add two binary numbers
    public static int[] addBinary(int[] A, int[] B) {
        int n = Math.max(A.length, B.length);
        int[] sum = new int[n + 1]; // Extra bit for carry
        int carry = 0;

        for (int i = 0; i < n; i++) {
            int aBit = i < A.length ? A[i] : 0;
            int bBit = i < B.length ? B[i] : 0;
            int total = aBit + bBit + carry;
            sum[i] = total % 2;
            carry = total / 2;
        }

        sum[n] = carry;
        return sum;
    }

    // Method to subtract two binary numbers (using 2's complement for subtraction)
    public static int[] subtractBinary(int[] A, int[] B) {
        int[] negB = negateBinary(B);  // Negate B to get 2's complement
        return addBinary(A, negB);
    }

    // Method to negate a binary number (2's complement)
    public static int[] negateBinary(int[] B) {
        int n = B.length;
        int[] negB = new int[n];
        int carry = 1;

        // Invert the bits and add 1
        for (int i = 0; i < n; i++) {
            negB[i] = (B[i] == 0) ? 1 : 0;
        }
```

```
    // Add 1 to the result
    for (int i = n - 1; i >= 0; i--) {
        int total = negB[i] + carry;
        negB[i] = total % 2;
        carry = total / 2;
    }

    return negB;
}

// Method to right shift a binary number (shift both accumulator and multiplier)
public static void rightShift(int[] A, int[] Q, int Qm) {
    int n = Q.length;
    int previous = Qm;  // Q-1, the extra bit
    for (int i = n - 1; i > 0; i--) {
        Q[i] = Q[i - 1];  // Shift Q to the right
    }
    Q[0] = A[n - 1];  // Set Q[0] to the sign bit of the accumulator
    for (int i = n - 1; i > 0; i--) {
        A[i] = A[i - 1];  // Shift the accumulator to the right
    }
    A[0] = previous;  // Set the most significant bit of A to Q-1
}

// Method to perform Booth's multiplication
public static int[] boothMultiplication(int[] M, int[] Q) {
    int n = M.length;
    int[] A = new int[n];  // Accumulator (initialized to 0)
    int[] Qm = new int[1];  // Q-1, initialized to 0
    Qm[0] = 0;

    int[] result = new int[2 * n];  // This will hold the final result (A + Q)

    for (int i = 0; i < n; i++) {
        // Check the pair (Q0, Q-1)
        if (Q[0] == 0 && Qm[0] == 1) {
            // Add M to A
            A = addBinary(A, M);
        } else if (Q[0] == 1 && Qm[0] == 0) {
            // Subtract M from A
            A = subtractBinary(A, M);
        }

        // Right shift A, Q, and Q-1
        rightShift(A, Q, Qm[0]);

        // Update Q-1
        Qm[0] = Q[n - 1];
    }
```

```java
    // Store the result in the final result array
    for (int i = 0; i < n; i++) {
        result[i] = A[i];
    }
    for (int i = 0; i < n; i++) {
        result[n + i] = Q[i];
    }

    return result;
}

// Method to print binary number
public static void printBinary(int[] bin) {
    for (int i = bin.length - 1; i >= 0; i--) {
        System.out.print(bin[i]);
    }
    System.out.println();
}

// Method to convert binary array to decimal value
public static int binaryToDecimal(int[] bin) {
    int decimal = 0;
    int power = 1;
    for (int i = 0; i < bin.length; i++) {
        decimal += bin[i] * power;
        power *= 2;
    }
    return decimal;
}

// Main method to test Booth's multiplication
public static void main(String[] args) {
    // Test with two 4-bit numbers (e.g., 6 and -3 in decimal)
    int[] M = {1, 1, 0, 0};  // M = 6 in decimal
    int[] Q = {1, 0, 0, 1};  // Q = -3 in decimal (in 2's complement)

    System.out.print("Multiplicand M = ");
    printBinary(M);
    System.out.print("Multiplier Q = ");
    printBinary(Q);

    // Perform multiplication using Booth's algorithm
    int[] product = boothMultiplication(M, Q);

    System.out.print("Product = ");
    printBinary(product);  // Output the binary product

    // Optionally, print the decimal result for verification
    int decimalProduct = binaryToDecimal(product);
    System.out.println("Decimal Product = " + decimalProduct);
}
```

```
}
● adityabhardwaj@Adityas-Air JAVA %
  ExceptionMessages -cp /Users/adity
  t.java/jdt_ws/JAVA_db575eb7/bin Bo
  Multiplicand M = 0011
  Multiplier Q = 1001
  Product = 00100001
  Decimal Product = 33
○ adityabhardwaj@Adityas-Air JAVA %
```

EXPERIMENT-8

Arithmetic Logic Unit

(ALU) Theory:

The Arithmetic Logic Unit (ALU) is a critical component of a CPU that performs arithmetic operations
(addition, subtraction, etc.) and logical operations (AND, OR, XOR, etc.). It accepts inputs, performs operations according to control signals, and outputs the result. ALUs are the core processing units in CPUs and are integral to all computations in a computer system.

Code:

```
#include <iostream>
using namespace std;

int ALU(int op, int a, int b) {     switch(op) {         case 0: return a + b; // Addition         case 1: return a - b; //
Subtraction         case 2: return a & b; // AND         case 3: return a | b; // OR         case 4: return a ^ b; // XOR
default: return 0;
    }
}

int main() {     int a = 5, b = 3;     cout << "Addition: " << ALU(0, a, b) << endl;     cout << "Subtraction: " << ALU(1, a, b) << endl;
cout << "AND: " << ALU(2, a, b) << endl;     cout << "OR: " << ALU(3, a, b) << endl;     cout << "XOR: " << ALU(4, a, b) << endl;
return 0;
}
```

Output:

```
Addition: 8
Subtraction: 2
AND: 1
OR: 7
XOR: 6
```

EXPERIMENT -9

Memory Design

Theory:
Memory design refers to the construction of storage systems within a computer. This includes various types of memory, such as RAM, ROM, and cache, each serving a different purpose.

The design process focuses on optimizing speed, capacity, and data access time. Efficient memory design is crucial for overall system performance, especially in handling large amounts of data.

Code:

```
#include <iostream>

int main() {
    // Simulate memory using an array     int memory[5] = {10, 20, 30, 40, 50};

    // Access and manipulate memory     std::cout << "First value in memory: " << memory[0] << std::endl;     std::cout << "Second value in
memory: " << memory[1] << std::endl;

    // Perform some operations     int result = memory[0] + memory[1];  // Adding first and second memory values     std::cout << "Result of addition: " << result <<
std::endl;

    return 0;
}
```

Output:

```
First value in memory: 10
Second value in memory: 20
Result of addition: 30
```

EXPERIMENT -10

Associative Cache Design

Theory: An associative cache is a type of memory cache where data can be stored in any location, as opposed to being mapped to a specific address. It uses a content-based approach to determine where data is stored and retrieved. The design typically involves cache tags and lookup tables to ensure fast access to data.

Code:

```
#include <iostream>
#include <vector>

class Cache { public:
    Cache(int size) : cache(size, -1) {}

    bool lookup(int address) {         for (int i = 0; i < cache.size(); ++i) {
if (cache[i] == address) {                return true;  // Cache hit
            }
        }
        return false;  // Cache miss
    }

    void store(int address) {         for (int i = 0; i < cache.size(); ++i) {             if (cache[i] == -1) {  // Empty slot
cache[i] = address;                std::cout << "Stored " << address << " in cache." << std::endl;                return;
            }
        }
```

```
    // If no empty slot, replace the first item (for simplicity)        cache[0] = address;        std::cout << "Replaced with " << address
<< " in cache." << std::endl;
  }

private:    std::vector<int> cache;
};

int main() {
  Cache cache(4);  // Cache with 4 slots

  int address = 0x123;    if (!cache.lookup(address)) {        std::cout << "Cache miss at address " << std::hex << address << std::endl;
```

Output:

```
Cache miss at address 123
Stored 123 in cache.
Cache miss at address 456
Stored 456 in cache.
```

EXPERIMENT -11

Direct Mapped Cache Design

Theory: A direct-mapped cache design maps each block of main memory to exactly one cache line. It is a simpler design compared to associative cache and offers faster access time, but can lead to cache misses if multiple memory locations map to the same cache line. Direct mapping is used to strike a balance between complexity and performance.

Code:

```
DirectMappedCache { public:
  DirectMappedCache(int size) : cache(size, -1) {}    bool access(int address) {        int index = address % cache.size();        if (cache[index] == address) {
std::cout << "Cache hit at address " << std::hex << address << std::endl;            return true;
    }
    std::cout << "Cache miss at address " << std::hex << address << std::endl;        cache[index] = address;  // Store the address in the cache        return false;
  }
private:    std::vector<int> cache;
};
int main() {
  DirectMappedCache cache(4);  // Cache with 4 lines    int address = 0x123;    cache.access(address);    address = 0x456;
cache.access(address);    address = 0x123;  // This should be a cache hit    cache.access(address);    return 0;
      }
```

Output:

```
Cache miss at address 123
Cache miss at address 456
Cache hit at address 123
```

EXPERIMENT -12

CPU Design

Theory: CPU design involves the creation of the central processing unit, which is the brain of the computer. The CPU executes instructions from memory and performs calculations. The design includes elements like the control unit, ALU, registers, and cache. Understanding CPU design is key to optimizing performance and energy efficiency. **z**

Code:

```
#include <iostream>

class SimpleCPU { public:     void load(int value) {        accumulator = value;
   }
   void add(int value) {        accumulator += value;
   }
   void subtract(int value) {        accumulator -= value;
   }
   void multiply(int value) {        accumulator *= value;
   }
   void displayResult() {        std::cout << "Result: " << accumulator << std::endl;
   }

private:     int accumulator = 0;
};

int main() {
   SimpleCPU cpu;

   cpu.load(5);     // Load 5 into the accumulator     cpu.add(7);      // Add 7 to the accumulator     cpu.multiply(3);  // Multiply accumulator by 3
   cpu.displayResult();  // Display result (should print 36)

   return 0;
}
```

Output:

```
Result: 36
```

EXPERIMENT -13

Mathematical Expressions

Theory: This experiment involves writing and evaluating mathematical expressions in assembly or a low-level programming language. Mathematical operations, such as addition, subtraction, multiplication, and division, are implemented using registers and arithmetic logic units. Understanding how these operations work at the assembly level is crucial for low-level programming and optimization.

Code:

```
#include <iostream>
Int main(){
Int result=(3+5)*(7-4);

Std::cout<<"Result of expression (3+5)*(7-4):"<<result;

Return 0;

}
```

Output:

```
Result of expression (3 + 5) * (7 - 4): 24
```

EXPERIMENT -14

File Operations

Theory: File operations in programming refer to the tasks of reading from and writing to files. In assembly or low-level languages, this involves using system calls or interrupt services to interact with file systems. This experiment focuses on basic file operations like opening, reading, writing, and closing files.

Code:

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
   // File writing     std::ofstream outFile("example.txt");     if (outFile.is_open()) {
outFile << "Hello, World!\n";         outFile << "This is a test file.\n";         outFile.close();
   } else {         std::cerr << "Unable to open file for writing!" << std::endl;
   }

   // File reading     std::ifstream inFile("example.txt");     std::string line;     if
(inFile.is_open()) {         while (getline(inFile, line)) {             std::cout << line <<
std::endl;
       }         inFile.close();
   } else {         std::cerr << "Unable to open file for reading!" << std::endl;
   }

   return 0;
}
```

Output:

```
Hello, World!
This is a test file.
```